

TP N°6: TDD - Test Driven Development



Hoja de Estilos - Estándares y convenciones a seguir para trabajar con las implementaciones a desarrollar

GRUPO 8

DOCENTES:

- Ing. Laura Covaro
- Ing. Cecilia Massano
- Ing. Constanza Garnero
- Ing. Judith Meles

INTEGRANTES:

- 87834 - Barrionuevo Natalia
- 80738 - Benjamin, Ian Nicolas
- 81699 - Benedetti, Angelo
- 84066 - Brito, Serena
- 71471 - Enrique, Walter Ignacio
- 98965 - Fernandez, Santiago
- 81705 - García, Valentina
- 85296 - Gomez Alameda, Romina Abigail
- 82397 - Sosa, Diego
- 82127 - Tarquinio, Angel Valentín

Front

1. Stack Tecnológico



2. Organización del código



Para la entrega de este proyecto, se tomó la decisión estratégica de consolidar todo el código —la estructura HTML5, los estilos CSS y la lógica de negocio de JavaScript— en un único archivo `index.html`.

Esta decisión representa un trade-off deliberado, donde se prioriza la simplicidad de la entrega y la portabilidad por encima de la Separación de Responsabilidades (SoC).

Para este proyecto, el foco principal de la metodología TDD se implementó en el **backend**, desarrollado en Python, donde reside la lógica de negocio crítica. Por lo tanto, la interfaz de frontend se concibió como un **cliente ligero (thin client)** o un **'simulador'** cuyo único propósito era proveer una interfaz humana para disparar poder simular la user story.

3. Librerías usadas

No se utiliza ninguna librería de JavaScript externa.

Se basa exclusivamente en:

Tecnologías Web Nativas (Vanilla JS): Se usa JavaScript puro para toda la lógica, la manipulación del DOM (creación y modificación de elementos HTML), el manejo de eventos y la comunicación con el *backend* a través de la función nativa `fetch()`.

4. Styling

El *styling* en el código es **CSS puro** (CSS3) integrado directamente en la etiqueta `<style>` del documento HTML.

Técnica	Ejemplos de Uso	Propósito
Diseño Responsive	<code>@media (max-width: 768px), .grid, .participant-form</code>	Asegura que el layout se adapte correctamente a pantallas de móvil y tablet, cambiando a un diseño de una sola columna.
Layout Moderno	<code>display: grid, display: flex, gap</code>	Utiliza CSS Grid para la estructura principal (<code>.grid</code>) y Flexbox para alinear elementos internos (como los horarios <code>.schedules</code> y los campos del formulario <code>.checkbox-group</code>).
Interacciones y Transiciones	<code>transition: transform 0.2s, input:focus, .btn:hover</code>	Añade animaciones sutiles a los botones (pequeño desplazamiento en <code>:hover</code>) y transiciones de color en los bordes de los inputs, mejorando la experiencia del usuario.

Feedback Visual de Estado	<code>.capacity-info,</code> <code>.capacity-warning,</code> <code>.alert-error</code>	Utiliza colores específicos y clases de utilidad para dar feedback inmediato sobre la capacidad o el éxito/error del formulario.
----------------------------------	--	---

5. Reglas de nombrado

1. Nomenclatura Principal (Kebab-Case)

Se utiliza el formato **kebab-case** (todo en minúsculas y separado por guiones) para todas las clases CSS. Esta es la convención más común y recomendada en el desarrollo web moderno:

- Ejemplos: `.activity-card`, `.form-group`, `.schedule-btn`, `.alert-success`.

2. Estructura Lógica de Nomenclatura

Aunque no es un BEM estricto, la nomenclatura sigue una estructura de **Bloque - Elemento - Modificador** que organiza las clases por su función y estado:

Tipo	Convención	Ejemplos en el Código
Bloque (Componente)	Define el contenedor principal o componente.	<code>.container</code> , <code>.header</code> , <code>.activity-card</code> , <code>.form-group</code> , <code>.btn</code>
Elemento (Child)	Clase descriptiva para una parte del bloque.	<code>.field-hint</code> (es un hint dentro de un field/form)

Modificador (Estado/Variante)	Indica un estado, variación o color alternativo.	<code>.schedule-btn.**selected**</code> , <code>.capacity-info</code> , <code>.capacity-warning</code> , <code>.alert-error</code> , <code>input.**touched**</code>
--------------------------------------	--	--

6. Misceláneas

El código JS se enfoca en la funcionalidad moderna del navegador:

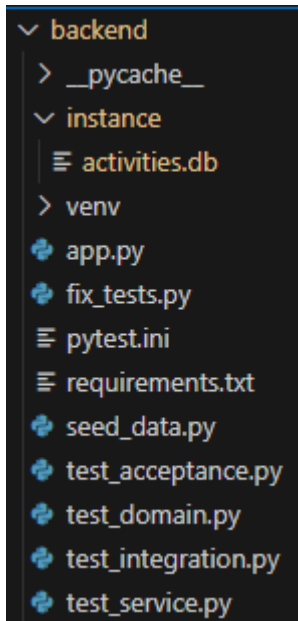
1. **Comunicación:** usa la función nativa `fetch()` y `async/await` para todas las llamadas a la API de Flask.
2. **Estado:** variables simples (`selectedActivity`, `selectedSchedule`) gestionan la interactividad.
3. **Validación:** implementa validaciones **frontend** con `pattern` (HTML) y la clase CSS `.touched` para dar feedback visual sobre errores.
4. **UX:** incluye la función `showAlert` para manejar mensajes de éxito/error con auto-ocultamiento.

Back

1. Nomenclatura

- **PascalCase (o UpperCamelCase):** se usa para nombrar todas las **clases** del código, incluidos los modelos de la base de datos y las clases de servicio. Por ejemplo: `Activity`, `Visitor` y `ActivityService`.
- **snake_case (o lowercase_with_underscores):** es el formato más común y se utiliza para nombrar funciones, métodos, variables, parámetros y los atributos de la base de datos. Por ejemplo: `get_activities`, `register_visitor`, `activity_id` y `available_capacity`.
- **SCREAMING_SNAKE_CASE:** se utiliza para definir las **constantes de configuración** a nivel de aplicación (variables que no cambian). Por ejemplo: `SQLALCHEMY_DATABASE_URI`.

2. Organización del código



backend/

- **app.py**
 - Punto de entrada principal de la API REST.
 - Contiene la definición de modelos ORM (`Activity`, `Visitor`, `Registration`), funciones de utilidad (`generate_time_slots`, `is_valid_slot`, `get_turn_capacity`, `get_min_age`), servicios (`ActivityService.register_visitor`) y las rutas Flask correspondientes (`/api/activities`, `/api/activities/<id>/register`, `/api/visitors`).
 - Se encarga de la inicialización de Flask, SQLAlchemy y CORS, configurando la base de datos `sqlite:///activities.db`.
- **seed_data.py**
 - Script independiente diseñado para la creación y precarga de la base de datos con datos de ejemplo (actividades y horarios).
 - Ideal para entornos de desarrollo.
- **requirements.txt**
 - Lista detallada de las dependencias del proyecto (ej., Flask, Flask-SQLAlchemy, Flask-CORS, pytest, etc.).
- **pytest.ini**
 - Archivo de configuración para pytest, incluyendo *fixtures* y *markers*.
- **test_*.py**
 - Conjunto exhaustivo de pruebas (unitarias, integración, aceptación) implementadas con pytest y pytest-flask.

3. Estructura de Funciones

Decoradores: se utilizan decoradores para mapear y registrar funciones como manejadores de solicitudes (funciones de vista), asociándose a una URL específica y a los métodos HTTP. Esto hace que la definición de rutas sea limpia y fácil de leer justo encima de la función que implementa la lógica.

4. Control de errores

Devolución de Errores en Formato JSON: el *backend* devuelve los mensajes de fallo en formato JSON (JavaScript Object Notation).

Uso de la Estructura `try-except`: para prevenir que un error cause la caída total del *backend* y asegurar que se pueda generar una respuesta JSON adecuada, se utiliza la estructura de control de flujo `try-except` (o `try-catch` en algunos lenguajes).

5. Comentarios y Documentación

1. Comentarios de Línea (`#`):

- Se utilizan para **aclarar** detalles de implementación que no son inmediatamente obvios (ej. `db.session.flush()` `# Para obtener el ID del visitante`).
- Sirven para **separar el código en bloques lógicos** (ej. `# Modelos`, `# Rutas de la API`).

2. Docstrings (`"""Docstring"""`):

- Funcionan como **comentarios de bloque** formales.
- Se utilizan inmediatamente después de la definición de una **clase** o **función** para describir su propósito y su responsabilidad principal (ej. la función `register_visitor` tiene un *docstring* que dice: `"Registra un visitante en una actividad"`).

6. Librerías

Componentes de la API (Backend)

- **Flask:** El **microframework web** principal; maneja rutas y peticiones HTTP.

- **Flask-SQLAlchemy**: Extensión para gestionar la **base de datos** (modelos de datos y transacciones).
- **Flask-CORS**: Extensión para permitir la **comunicación** entre el frontend y el backend (evita bloqueos de seguridad).

Pruebas y Desarrollo

- **pytest**: **Framework de pruebas** estándar para escribir y ejecutar tests.
- **pytest-flask**: Plugin que facilita las **pruebas específicas** de rutas y funcionalidad de Flask.
- **python-dotenv**: Herramienta para cargar **variables de configuración** (secretos, URL) desde un archivo **.env**.